

Offline Legal Translation — Operating Manual

Version 1.4 · 2026-09-10 · EN↔SL, EN↔DE · hpelitebook8g1i16, Ubuntu 26.04

This document consolidates and replaces the Hardware Assessment, Workflow Architecture, and Addenda 1 and 2. Version 1.3 revises it for the multi-project confidential layout, the encrypted container, and the Claude Code project location; version 1.4 adds English↔German. Everything currently believed correct is here.

1. What this system is

A local pipeline that produces draft translations of legal documents on one machine, with no network egress, so that a certifying translator edits rather than translates from scratch. It is judged on one number only: whether editing a machine draft takes less time than translating the same page fresh.

Everything is arranged around that measurement. Section 6 is the experiment that decides whether to proceed; nothing beyond it should be built until that number is known.

1.1 The commands you will use most

Command	Purpose
tr-project	List, create, or switch the active confidential project
tr-inventory	Classify every file in source/ by source language
tr-status	What is translated, what is missing, what is stale
tr-run	Translate everything in source/ not yet in translated/
tr-lint	Deterministic checks; produces the reviewer worklist
tr-xlsx --survey	Measure spreadsheet redundancy before committing to it

Command blocks are colour-coded: **command** *--option* \$VARIABLE "string" # comment

2. Layout and naming

Confidential work lives in one encrypted container holding many projects. Each project is self-contained: its own sources, translations, working files, logs, and translation memory. Filenames are preserved exactly from source to output, so finding gaps is a plain set difference between two directories.

```
~/Claude_Stuff/cli_projects/
translation-tools/          the kit. Outside the data tree.
  bin/ lib/ glossary/ prompts/
~/.translate-venv/          python env. Survives unmount.

~/translation-work/
confidential-projects/      <- ENCRYPTED CONTAINER MOUNTS HERE
  .active                  which project the tools operate on
  _shared/
  glossary/base.tsv        terminology reusable across matters
```

glossary/nontranslatable.txt	
prompts/translate.txt	
kranj-2024/	a project
source/	originals. Read-only.
translated/	deliverables. SAME filenames.
work/ocr/	OCR output and text layers
work/tm.sqlite	memory, cache, and resume state
logs/	
glossary/project.tsv	case-specific terms; override base
project.conf	language pair, suffix, OCR languages
maribor-2025/	another project, fully isolated

What is shared and what is not. Glossaries layer: the shared base supplies settled legal terminology, and each project overlays case-specific renderings that win on conflict. Translation memory does NOT layer. Each project keeps its own, because memory holds actual source and target sentences, and those must not cross between matters with different clients and different confidentiality obligations.

Subdirectories under a project's `source/` are mirrored into `translated/`. If a client permits the suffix variant, set it in that project's `project.conf` and every tool follows:

```
TR_SUFFIX=_translated      # in <project>/project.conf
```

2.1 The \$KIT shorthand

The kit path is long, so this manual uses `$KIT` throughout. Define it once and the command blocks below can be copied verbatim.

```
echo 'export KIT=~/.Claude_Stuff/cli_projects/translation-tools' >> ~/.bashrc
source ~/.bashrc
```

The kit deliberately sits on a different branch of the filesystem from the case data. The Claude Code guard refuses to start a session from anywhere inside or above `~/translation-work/`, so the scripts must live outside that tree for `cd $KIT && claude` to be a safe operation. Keeping them under `Claude_Stuff/cli_projects/` also puts all Claude Code work in one place.

No script in the kit hardcodes its own location; each derives it at run time, and `core.hooksPath` is relative, so both travel with the directory. Two lines in `~/.bashrc` do not: the `export KIT=` and the `$KIT/bin PATH` entry `tr-setup` wrote with the old absolute path. Re-running `tr-setup` after a move appends the new `PATH` line but leaves the stale one, so edit `~/.bashrc` by hand.

2.2 Where these documents live

Keep the reference documents at `~/translation-work/docs/`, one level above the container — not inside `confidential-projects/`.

The reason is that `confidential-projects/` is a mount point. When the container is closed that directory is empty by design, so documents stored there vanish exactly when they are most needed: at the moment you are trying to remember how to open the container. Documents kept one level up remain available whether the container is open or closed, survive recreating

it, and contain no case material, so nothing is lost by holding them outside the encrypted volume.

```
mkdir -p ~/translation-work/docs  
# then save the manual, handover, and container documents there
```

The Claude Code deny rules are scoped to confidential-projects/ rather than all of ~/translation-work/, so a session can still read these documents while remaining unable to read any case data.

2.3 Selecting a project

Every tool operates on the active project. This is the one new way the layout can go wrong, so the tools are deliberately noisy about it: tr-run prints a banner and asks for confirmation before touching anything, and any tool refuses outright when no project is set.

```
tr-project                # show active, list all with file counts  
tr-project kranj-2024      # switch  
tr-project --new maribor-2025 # scaffold and switch  
tr-project --none         # clear  
  
# override for a one-off, without changing the active project  
TR_ROOT=~/translation-work/confidential-projects/kranj-2024 tr-status
```

The active project is recorded in a file inside the container, not in your shell, so it persists across terminals and is unset automatically whenever the container is closed.

3. One-time setup

Run these once, in order, from your home directory. Total time is dominated by the model download.

3.1 Install the kit

```
# a shorthand worth putting in ~/.bashrc, used throughout this manual
export KIT=~/.Claude_Stuff/cli_projects/translation-tools

mkdir -p "$KIT"
git clone git@github.com:hmbseaotter/offline-legal-translation.git "$KIT"
chmod +x "$KIT"/bin/*
"$KIT"/bin/tr-setup
source ~/.bashrc           # picks up PATH and TR_PROJECTS
```

tr-setup installs OCR and PDF tooling, builds a Python virtual environment outside the container so it survives unmounting, and warns if swap is too small. It creates no project data — that comes from case-init and tr-project. It is safe to re-run.

Then the container and the guard, both covered in full by the Encrypted Case Container document:

```
cd "$KIT"
./bin/case-init 40G           # LUKS2 container, once

cp bin/case-guard ~/.local/sbin/claude # refuses to start while mounted
chmod +x ~/.local/sbin/claude

case-open
tr-project --new kranj-2024
```

3.2 Increase swap

Do this before any long run. It is a safety net against the out-of-memory killer terminating a multi-day job, not a performance measure; inference should never actually touch swap.

```
sudo swapoff /swapfile
sudo falloca -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
free -h           # expect 8.0Gi swap
```

3.3 Install and register the model

GaMS3-12B-Instruct is a Slovene-adapted model from the University of Ljubljana, built on Gemma 3. It is not in the Ollama library, so it comes from Hugging Face as a GGUF. Q8_0 is chosen because quantization damage falls hardest on exactly the sparse morphological patterns Slovene depends on, and throughput is not a constraint here.

```
tr-model           # ~13 GB download, once

# equivalent to:
```

```
# ollama pull hf.co/mradermacher/GaMS3-12B-Instruct-GGUF:Q8_0
# ollama create gams3:q8 -f <Modelfile pinning num_ctx and temperature>

ollama list           # confirm gams3:q8 is present
```

The alias matters. Scripts refer to models by alias, chosen by language pair in one table in `lib/trlib.py` — `gams3:q8` for Slovene ↔ English, `euollm9b-2512:q8` for English ↔ German — so switching a model later is one `tr-model` invocation and one line in that table, rather than an edit in eight places. It also pins `num_ctx` and `temperature` so behavior does not drift with an Ollama upgrade.

English ↔ German needs its own model, because German is absent from GaMS3's training. Register it only on a machine that will take such a matter: `tr-model hf.co/mradermacher/EuroLLM-9B-Instruct-2512-GGUF:Q8_0 euollm9b-2512:q8` (about 10 GB, once).

Verify the tag at pull time. The repository publishes both static and imatrix-weighted quantizations and tag syntax occasionally changes. If `tr-model` fails, open the model page, copy the exact tag, and pass it as the first argument: `tr-model hf.co/...:Q8_0`

3.4 Confidentiality hardening

The model makes no network calls, but two exposures remain and both are more likely to cause a breach than the inference stack.

Confirm the disk is encrypted. Case material on a laptop is exposed by theft, not by the network:

```
lsblk -o NAME,FSTYPE,MOUNTPOINT | grep -i crypt
```

If that returns nothing, the installation is unencrypted. Retrofitting is disruptive; an encrypted container holding only `~/translation-work/confidential-projects` — which is what `case-init` creates — is the pragmatic alternative.

Confirm no cloud-routed models. Recent Ollama versions can dispatch to hosted models, which would send document text off the machine:

```
ollama list           # no "-cloud" tagged entries
ollama ps            # what is actually loaded right now
```

3.5 Verify the install

```
echo "Obdolženi je bil obtožen po čl. 211 odst. 2 KZ-1." \
| ollama run gams3:q8 --verbose
```

Read two things from the `--verbose` output: `prompt eval rate (prefill)` and `eval rate (generation)`. Measured on this hardware with `gams3:q8`, over 12 samples of real pipeline traffic: `prefill 7.00 tokens per second`, `generation 2.20`. What governs planning is neither of those but the sustained rate — `0.81 output tokens per second`, `48.3 seconds of wall clock per segment`. The earlier estimate of 4 tokens per second was optimistic by roughly five times,

because most of a segment's cost is not generation: every call re-reads a 200-token system prompt at 7 tokens per second, some 28 seconds, and pays it whether the segment is a sentence or two words. A faster quantization therefore attacks the smaller half of that cost; batching several segments into one call, or shortening the system prompt, attacks the larger.

4. Before bulk translation

Four things to settle before the corpus is processed. The first decides which files are in scope at all. The other three determine output quality far more than any model setting, and none of them can be automated.

4.1 Triage — which files are in scope

A client drop is not a clean corpus. Files arrive in the folder structure the client used — often two or more separate drops, nested several levels — and that structure is carried through source/ to translated/ untouched. The tree also mixes languages: Slovene alongside English, Croatian or Serbian, Armenian, and whatever else a matter happens to contain. Only the files in the project's source language are translated; the rest are counted and set aside.

tr-inventory walks source/ in full, records every file whether or not the pipeline can convert it, and detects the language of each. Run it before tr-status or tr-run — both work from its manifest.

```
tr-inventory      # classify every file in source/
tr-inventory --rescan  # re-examine files already recorded
tr-inventory --no-ocr # skip the sampling pass on scanned PDFs
```

This is not tidiness. A Croatian file pushed through a Slovene-to-English prompt costs hours of inference at 48 seconds a segment and produces confident nonsense — and the result is written into work/tm.sqlite keyed on the source text, so it is reused silently every time that segment reappears. Undoing it means editing the memory or retranslating the matter. tr-run warns rather than proceeding quietly when no inventory exists.

Three files are written to work/inventory/. manifest.tsv lists every file with its path, format, detected language and confidence. by-lang/ splits those paths into one list per language. summary.txt holds counts and no paths at all, and is the only one of the three that may be sent to the client — a filename in a criminal matter routinely carries a party name, a date or a case number.

Scanned PDFs have no text layer to detect from, so tr-inventory OCRs a page or two with the candidate languages combined and decides from that; the full OCR pass in S5.3 then runs in the right language rather than a guessed one. Anything it cannot place — too little text, an unsupported format, an unreadable file — goes to the undetermined bucket with its path, for a human to deal with. Nothing is guessed.

4.2 The glossary — highest leverage

Two files, layered. The shared base holds legal terminology that carries across matters; the project file holds case-specific renderings and wins on conflict. The certifying translator fixes each once, up front, and every later deviation is reported by tr-lint with its location.

```
P=~/.translation-work/confidential-projects

nano $P/_shared/glossary/base.tsv      # reusable across cases
nano $P/kranj-2024/glossary/project.tsv # this matter only
```

```
# obdolženi<TAB>the accused<TAB>criminal procedure
# oškodovanec<TAB>the injured party
# zaslišanje<TAB>examination<TAB>not "hearing"
```

Promote a term to the base file only once it is settled and not case-specific. Names, entities, and matter-specific usages stay in the project file.

Extract candidates from the corpus rather than inventing them. Charges, statutory references, court and agency names, procedural terms, party designations, document types. The already-installed Qwen model is fast and well suited to drafting the candidate list for the translator to rule on.

4.3 Non-translatables

Regex patterns in nontranslatable.txt mark strings reproduced verbatim: case numbers, file references, statute short forms. Defaults for Slovene court formats are supplied. Every one of these the model would otherwise “helpfully” translate is a guaranteed edit at a known location, so catching them in code is free quality.

Dates, amounts and times are NOT in that class. They are converted, because that is what a translation is. Into English, 5. 3. 2024 becomes March 5, 2024; 12.450,00 becomes 12,450.00; a 24-hour time becomes 2:30 p.m. Into Slovene it runs the other way: 5. marec 2024 - the day carries a period because it is an ordinal, so bare “5” would read as the cardinal number rather than the fifth day; the month name is lowercase, which Slovene requires; and the three parts are spaced. Amounts take 12.450,00 and times stay on the 24-hour clock. The value never changes; only its spelling does. Two forms are wrong in either language and worth naming, because both were in this kit before the translator corrected them: “5 March 2024” and “5. Marec 2024”. The all-numeric 5.3.2024 is technically correct Slovene but invites the reader to wonder whether it means 5 March or 3 May, which is a poor property for evidence.

The distinction has a practical edge. Inside a sentence the model does the conversion. A segment that is only a date or an amount never reaches the model at all, because the patterns above exclude it from translation - so a Datum column in a spreadsheet stayed in Slovene while the same date in prose came out in English, and the deliverable contradicted itself column by column. Those whole-segment values are now converted in code: exact, consistent, and without the 48 seconds a model call would cost.

```
nano $P/_shared/glossary/nontranslatable.txt

# test a pattern before trusting it (kit dir, no case data needed):
cd "$KIT" && python3 -c "
import sys; sys.path.insert(0,'lib'); import trlib
print(trlib.is_translatable('I K 12345/2024')) # expect False"
```


4.4 Reuse the translator's prior work

The metric is edits this particular translator makes. A draft that matches their established renderings and register produces fewer corrections than an objectively similar draft that does not, so earlier translations are probably the single highest-value input available.

They go in the project's reference/ folder, one language a side, paired by path: reference/en/leases/lease-2023.docx beside reference/de/leases/lease-2023.pdf. Each side may be Word, PDF with a text layer, scanned PDF or plain text, whatever the other side is. tr-ref lines each pair up sentence by sentence without a model — by length, with the numbers both sides share as anchors — and keeps only one-to-one pairs whose numbers agree; a sentence with no number is kept only where its neighbours lined up one-to-one too. tr-run then gives an identical source sentence the translator's rendering instead of a draft, and tr-terms --reference proposes the glossary from those renderings.

Everything kept is listed in work/reference/pairs.tsv, to be read before the first run, because every line there can reach a deliverable word for word. Two kinds are never reused: a sentence the references translate differently (tr-ref --conflicts), and a translation read by OCR, whose misreadings would pass straight into the output — those count for terminology only. References stay in their project: memory never crosses matters, and a reference is a client's document.

On invented test documents a Word pair kept 12 of 14 sentence pairs and a Word-to-PDF pair 4 of 6; the rest were rejected rather than guessed. A reused sentence is not checked by tr-lint: it is the translator's own work, and it never enters the translation memory. Harvesting terminology from references needs volume — on seventeen pairs tr-terms --reference proposes mostly noise.

5. Handling each source type

5.1 Word documents

```
cd ~/translation-work/confidential-projects/kranj-2024
tr-docx source/ovadba.docx translated/ovadba.docx --from sl --to en
```

Paragraphs, headings, lists, tables, headers and footers are preserved. Each paragraph is split into sentences before translation, so the memory keys on sentences and repeats are reused across the whole corpus.

Known limitation. Formatting that varies within a single paragraph — one bold word mid-sentence — is not preserved; the paragraph takes the formatting of its first run. Paragraph-level styling is intact. If intra-paragraph emphasis is legally significant in this corpus, use OmegaT for those files instead (Section 9).

5.2 Spreadsheets

Always survey first. Large tables are dominated by repeated categorical values, and deduplication both collapses the work and guarantees identical cells receive identical translations.

```
# measure first, translate nothing
tr-xlsx source/tabela.xlsx --survey

# then translate; each distinct string is translated once
tr-xlsx source/tabela.xlsx translated/tabela.xlsx

# restrict to specific columns if only some hold text
tr-xlsx source/tabela.xlsx translated/tabela.xlsx --cols B,D,E
```

The survey prints the unique-string ratio and an estimate of translation time with and without deduplication. On the 7,000-row table, run this before planning anything around it. Output stays in XLSX, which is correct — ten columns will not transfer usefully into Word.

5.3 Scanned PDFs — two stages, deliberately

This is the highest-risk path in the system, for a specific reason: a model fed corrupted OCR does not flag the corruption. It produces a fluent, confident translation of the corruption. A misread digit in a date or an amount arrives looking entirely correct.

```
# stage 1 - OCR only, then stop
tr-pdf --ocr-only source/zapisnik.pdf

# read the text layer against the page images
less work/ocr/zapisnik.txt
xdg-open work/ocr/zapisnik.ocr.pdf

# stage 2 - only after verification
tr-pdf source/zapisnik.pdf translated/zapisnik.docx
```

Check names, dates, case numbers and amounts against the page. Diacritics matter less than they look: a dropped caron usually leaves a non-word that Slovene spell-check flags, and the residue is the handful of pairs that are both real words. Numbers have no such safety net — a misread digit is a perfectly well-formed token, so nothing downstream will ever question it. Words Tesseract could not read at all are already marked OCR_ILLEGIBLE by tr-ocrtext, which keeps the per-word confidence that pdftotext throws away. OCR output is cached, so stage 2 does not re-run it.

A second engine is the only check on a misread digit. A vision model reads a page by a completely different mechanism than Tesseract, so their failure modes are largely uncorrelated: where both produce the same number it is almost certainly right, and where they differ one of them is wrong. ocr-check.py runs both over the same pages and reports only counts — never document text — so its output is safe to discuss outside the container.

Measured on real evidence. deepseek-ocr:3b against Tesseract over eight pages of two scanned prosecution documents. Diacritics agreed almost exactly (64/64, 72/72, 83/83 on the

pages carrying most of them), which settles whether a 3B model reads Slovene. Numbers are where it earns its keep: on seven pages the two engines agreed on every number, and on the eighth — the one page also carrying 27 low-confidence words and four hand-filled digit tokens — they disagreed on fourteen, none explainable as a formatting difference.

It costs about 3 minutes a page against seconds for Tesseract, so it is affordable on the pages that carry the risk and not across a whole corpus. That is what `--gate` decides: the vision pass runs only where the share of Tesseract words below the confidence floor reaches the threshold. The default of 3% comes from those eight pages — the page with fourteen disagreements sat at 5%, the clean ones at 0–2%. Two points is thin evidence; raise it if the second engine keeps confirming the first.

The third criterion is deliberately not a rate. A page can be 99% clean and still carry one wrong digit in an amount, and nothing downstream can catch it, so a disagreement is not a reason to stop the project - it is a page that gets read against the original. The first two thresholds come from eight pages of two real documents and are starting points with a stated basis, not laws. The Phase 1 files measured 1% and 3%.

`tr-ocrstat` answers this for a whole corpus rather than one file: it reports the unreadable-token rate per PDF, worst first, against the thresholds below, and exits non-zero if anything is in the stop band or any text layer cannot be measured. Use it before `tr-run`; use `ocr-check.py` on the individual pages whose numbers carry weight.

A text layer written by an earlier `tr-inventory --count --with-ocr` cannot be measured. That version made it with plain `pdftotext`, so nothing in it is marked `OCR_ILLEGIBLE`, and `tr-pdf` reused it for translation. Such layers are recognisable — `pdftotext` leaves a form feed after every page, which neither `tr-ocrtext` nor the born-digital path writes — so `tr-ocrstat` names them rather than reporting 0%, and `tr-pdf` reads the file again, keeping the old layer as `<name>.txt.unmarked`. `tr-inventory --with-ocr` now has `tr-pdf` make every text layer.

What counts as too poor to translate. Not a judgement call, because the measurements already exist. `tr-ocrtext` reports the share of tokens Tesseract scored below its confidence floor; `ocr-check.py` reports the share of doubtful words and the number disagreements between the two engines. Unreadable tokens: under 5% proceed, 5-20% inspect, 20% or more stop. Doubtful words: under 3% proceed, 3-10% run the cross-check, over 10% stop. Number disagreements: any at all, and a person reads that page.

The earlier figure of 6.7 minutes a page, and the caveat that only a clean render had been tested, both belonged to `qwen3.6` and are superseded. The smaller purpose-built model is roughly thirty times faster on the same page and has now been run on real scans, skew, stamps and handwriting included.

```
ocr-check.py source/zapisnik.pdf --pages 2 # both engines, then compare
```

6. Phase 1 — the experiment that decides everything

Do not build further until this number exists. The comparison is deliberately simple.

Step	Action	Detail
1	Pick six comparable pages	Three for machine assistance, three matched for scratch translation. Same document type, similar density
2	Draft the three	tr-run, or tr-txt --bilingual for side-by-side review
3	Time the translator on both sets	Minutes per page, MT-assisted versus from scratch
4	Compute the ratio	This is the decision. Anything at or above 1.0 means stop
5	Ask which errors cost the time	Terminology? Register? Numbers? That tells you what to fix next

```
case-open && tr-project kranj-2024
cp /path/to/test/*.docx \
~/translation-work/confidential-projects/kranj-2024/source/
tr-status
tr-run
tr-lint

# bilingual layout for side-by-side review (source in grey above target)
tr-txt work/ocr/sample.txt /tmp/sample-bilingual.docx --bilingual
```

Step 5 matters as much as step 4. A ratio slightly above 1.0 caused by one fixable error class — say, untranslated case numbers — is a different verdict than the same ratio caused by systematically wrong register.

tools/phase1-setup.sh runs the whole sequence rather than leaving it to be reassembled from these steps. It expects two sets of three pages prepared beforehand — source/phase1/mt/ to be machine-drafted and source/phase1/scratch/ translated cold — and it refuses to continue past the OCR stage until the text layer has been looked at, because on a corpus of scans an unverified draft measures Tesseract rather than the tooling. The two sets must not share content: whichever is done second is faster for having been read already, and the ratio would record that instead.

```
tools/phase1-setup.sh
```

Record the timings in tools/phase1-tally.tsv, which holds minutes per page and a reason for each edit — terminology, register, numbers, additions, OCR — and no document text, so it can be discussed outside the container.

7. Daily operation

```
case-open      # unlock the container
tr-project     # confirm which project is active
tr-inventory   # classify the drop; after new files arrive

tr-status      # what remains
tr-status --missing # bare paths, for piping
tr-status -v    # include the completed list
```

```

tr-run -n          # dry run: what would be processed
tr-run           # everything missing
tr-run source/one.docx # a single file

tr-lint          # after any run

case-close       # lock up when finished

```

Closing does not delete anything. case-close unmounts the container; it does not empty it. Every source, draft and memory entry is still inside the container file and returns unchanged at the next case-open — the mountpoint only looks empty because nothing is mounted there. The other half of that is retention: a finished matter stays in the container at full size until someone opens it and deletes the project directory by hand. No script removes client work, so deciding when a matter should stop being held is a manual step (container document S4.5).

tr-run is resumable at two levels. It skips files already present in translated/, and within a file every segment already in the memory is reused rather than regenerated. Interrupting it costs at most one segment. Re-running after editing the glossary is cheap for the same reason.

A file is re-translated if its source is newer than its output, or if the existing output was produced by a different model or a superseded prompt. tr-run records which model and prompt version wrote each deliverable in work/deliverables.tsv and compares them on the next run, printing a redo line that names both. The prompt version is not typed by anyone: it is derived from the prompt text each language pair is actually sent, so editing prompts/translate.txt gives the pairs whose text changed a new version and leaves the others alone. The Slovene ↔ English text in use since v6 keeps that name. That condition was missing until it mattered: the prompt version is part of the memory's key, but tr-run skipped the whole file on mtime alone, before the memory was consulted, so a bump changed nothing. A prompt correction followed by a re-run reported three files skipped in three seconds and left the defective drafts in place. To force a redo, delete the output. To force a redo ignoring the memory, change TR_PROMPT_VERSION, which is part of the cache key:

```

rm translated/ovadba.docx && tr-run # reuse memory
TR_PROMPT_VERSION=redo-2026-08 tr-run # any unused value; ignore memory

```

8. The lint report

tr-lint runs no model, takes seconds, and catches the error classes a language model is structurally worst at noticing. Its output is not a pass/fail gate; it is a prioritized worklist to hand the translator alongside the drafts.

Check	Meaning
FAIL	The segment errored out and contains no translation. Fix and re-run first
NUM	A number in the source is absent from the target, or a number appears that was not in the source. Highest consequence class in legal evidence
NONTR	A string marked non-translatable was altered or dropped

Check	Meaning
INCON	The same source sentence was translated two different ways. A corpus-level finding no per-document review will surface
GLOSS	An agreed glossary term was not used
ECHO	Target identical to source — possible untranslated passthrough
EMPTY	Empty target

tr-lint
tr-lint --tsv work/for-translator.tsv
open the report in a spreadsheet for the reviewer
libreoffice --calc work/lint-report.tsv

Findings are sorted by severity, so the top of the file is where to start. Numeric comparison normalizes separators, so 1.234,56 and 1,234.56 are treated as the same number, and it folds locale conversions together as well: a month that became a word and an hour that shifted by twelve are the same value written differently, not a missing number. Without that, every date in the corpus raised a finding, and legal documents are made of dates.

8.1 The error the linter cannot catch

Give the model a source that quotes a provision and stops short of the famous continuation, and it finishes the sentence from memory. Asked to translate “everyone is entitled to a fair hearing before an independent and impartial tribunal”, it returns that plus “established by law” - three words the source does not contain. That is an interpretation presented as a translation, and in a certified document it is the translator's name on it.

Two properties make this the worst class of error in the system. It reads perfectly, so nothing draws the eye to it. And it carries no number, no glossary term and no non-translatable fragment, so no deterministic check can see it - every check in the table above passes.

Nor is it a prompt problem. Four prompt variants were tried, including one whose rule named the failure and explained why it is wrong; the model complied with everything else in that prompt and completed the provision anyway. Measured over ten well-known provisions, two were completed - both ECHR article 6, the most quoted text in criminal procedure.

What can be detected is the context. Any segment citing a statute or treaty is flagged, and those are the segments to read against the source word by word. Asking the model to audit its own output against the source caught both real additions with no false alarms, at about 12 seconds a segment - a second opinion is available if wanted, and applied only to flagged segments it costs a few percent of a run rather than several times it.

The gap that remains: a famous provision paraphrased without a citation marker is not flagged, and nothing detects it.

9. When to use OmegaT instead

The scripts above are the fast path: they produce deliverables directly, with filenames preserved, which is what the client wants. They are the right tool for Phase 1 and for the spreadsheet.

OmegaT is the better container for sustained production work, for three reasons: it preserves intra-paragraph formatting properly, it gives the translator a sentence-by-sentence review interface with source and target aligned, and it accumulates a translation memory they own and can reuse on later matters.

The two coexist. Pre-translate with these scripts, export the memory as TMX, and import it into an OmegaT project so segments arrive pre-populated. That path requires no plugin and works with any CAT tool the translator prefers, including Trados.

```
sudo apt install omegat

# export the accumulated memory as TMX for import
cd "$KIT" && python3 -c "
import sys,sqlite3,html; sys.path.insert(0,'lib'); import trlib
db=sqlite3.connect(trlib.path('work','tm.sqlite'))
rows=db.execute('SELECT src,tgt,direction FROM tm').fetchall()
print('<?xml version=\'1.0\' encoding=\'UTF-8\'?>')
print('<tmx version=\'1.4\'><header srclang=\'sl\' '
      'segtype=\'sentence\' datatype=\'plaintext\'><body>')
for s,t,d in rows:
    a,b=d.split('-')
    print(f'<tu><tuv xml:lang=\'{a}\'><seg>{html.escape(s)}</seg></tuv>'
          f'<tuv xml:lang=\'{b}\'><seg>{html.escape(t)}</seg></tuv></tu>')
print('</body></tmx>')" > work/export.tmx
```

The TMX export above is a sketch and should be validated against OmegaT before relying on it.

10. Long unattended runs

A job running for days has failure modes a short one does not.

```
# prevent suspend from silently halting the run
systemd-inhibit --what=idle:sleep:handle-lid-switch \
               --why="translation batch" tr-run

# or run detached and reattach later
sudo apt install tmux
tmux new -s tr
tr-run          # then Ctrl-b d to detach
tmux attach -t tr

# monitor from another terminal
watch -n5 "sensors | grep -i 'Package id'; free -h | head -2; ollama ps"
tail -f "$(tr-project | awk '/^active/{print $2}')" /logs/run-*.log
```


Keep the machine on mains power and elevated for airflow. A 15 W mobile processor at sustained full load for days is outside its normal duty cycle. If the firmware exposes a charge limit, cap it so the battery is not held at full charge continuously:

```
cat /sys/class/power_supply/BAT0/charge_control_end_threshold
echo 80 | sudo tee /sys/class/power_supply/BAT0/charge_control_end_threshold
```

11. Troubleshooting

Symptom	Cause and fix
no active project	Run <code>tr-project <name></code> . Tools refuse rather than guess
container not mounted	Run <code>case-open</code> . When closed the path is empty by design
Wrong project translated	<code>tr-run</code> prints a banner and asks first; read it. Delete the wrong outputs and re-run
Connection refused	Ollama is not running. <code>systemctl status ollama</code> ; <code>sudo systemctl start ollama</code>
[TRANSLATION FAILED] in output	Model unreachable or timed out. Fix the cause, then re-run; cached segments are not regenerated
Process killed mid-run	Out of memory. Check swap (S3.2) and lower <code>TR_NUM_CTX</code>
Output contains commentary	Model ignored the output-only instruction. Lower temperature, or tighten prompts/ <code>translate.txt</code>
Sentences split at abbreviations	Add the abbreviation to <code>ABBREV</code> in <code>lib/trlib.py</code> and re-run with a new <code>TR_PROMPT_VERSION</code>
Case numbers being translated	Add a pattern to <code>nontranslatable.txt</code> ; verify with <code>is_translatable()</code> per S4.3
Very slow	Confirm mains power and that the platform profile is not power-saver: <code>cat /sys/firmware/acpi/platform_profile</code>
Diacritics wrong in OCR	Confirm <code>-l slv+eng</code> was used. Set <code>TR_OCR_LANGS</code> if the pair differs

12. Environment variables

Variable	Default	Purpose
<code>TR_PROJECTS</code>	<code>~/translation-work/</code> <code>confidential-projects</code>	Container root holding all projects
<code>TR_ROOT</code>	(active project)	Override to target one project for a single command
<code>TR_MODEL</code>	(by language pair)	Overrides the model chosen for the pair: <code>gams3:q8</code> for <code>sl↔en</code> , <code>eurollm9b-2512:q8</code> for <code>en↔de</code> , none for <code>sl↔de</code> . Set it in a project's <code>project.conf</code> , not in <code>~/.bashrc</code>
<code>TR_SRC</code> / <code>TR_TGT</code>	<code>sl / en</code>	Per-project, in <code>project.conf</code> . Any two of <code>sl</code> , <code>en</code> , <code>de</code>
<code>TR_SUFFIX</code>	(empty)	Per-project, in <code>project.conf</code> . Set if the client requires it
<code>TR_NUM_CTX</code>	8192	Context window. Lower if memory is tight

Variable	Default	Purpose
TR_PROMPT_VERSION	(from the prompt text)	Override only. Derived per pair from the prompt text actually sent, so editing the prompt changes it; sl↔en is v6
TR_OCR_LANGS	slv+eng	Tesseract languages of the source documents: eng for an English drop, deu for German
TR_OLLAMA	http://127.0.0.1:11434	Ollama endpoint
TR_DICTS	/usr/share/hunspell	Where tr-inventory looks for the hunspell word lists it detects language with
TR_OCR_SAMPLE_LANGS	slv+hrv+eng	Tesseract languages for the detection sampling pass on scanned PDFs
TR_VENV	~/translate-venv	Python environment the scripts re-exec into. Set before tr-setup to put it elsewhere
TR_NO_REEXEC	(unset)	Set to 1 to stay on the system interpreter. Diagnostics only; imports will fail
CASE_IMG	~/case/confidential.luks	The LUKS container file. Read by case-init, case-open, case-status
CASE_MAP	casedata	Device-mapper name while the container is unlocked
TR_VISION_MODEL	deepseek-ocr:3b	Second OCR engine used by ocr-check.py. qwen3.6 is the fallback
TR_VISION_PROMPT	Extract the text in the image.	Prompt for that model. It transcribes; it does not follow instructions
TR_OCR_MIN_CONF	40	Tesseract confidence floor in tr-ocrtext. Below it, a word is marked unreadable
TR_ILLEGIBLE_MARK	OCR_ILLEGIBLE	What tr-ocrtext writes in place of a word it could not read
CLAUDE_DESKTOP_BIN	/usr/bin/claude-desktop	The real binary case-guard-desktop launches once it has checked the mount
CASE_MNT	~/translation-work/confidential-projects	Where the container mounts. Also what the claude guard checks

For an English→German matter, set the pair once in that project's project.conf rather than on the command line, so every later run inherits it:

```
tr-project --new example-en-de
nano ~/translation-work/confidential-projects/example-en-de/project.conf

TR_SRC=en
TR_TGT=de
TR_OCR_LANGS=eng
```

Nothing else is set: the model follows the pair. GaMS3 translates Slovene↔English; EuroLLM-9B-Instruct-2512 (Q8_0) translates English↔German, because German is absent from every stage of GaMS3's training. tr-run names the model in its banner and refuses to start when it is not installed. Slovene↔German has no model chosen and refuses to translate.

Measured on this machine with invented English legal text: about 10 seconds a segment over 42 requests, none failed, against 48 for GaMS3. Unlike GaMS3, most of that is generation, at 3 tokens a second, rather than reading the prompt, so batching short segments saves less. Two cautions from the same runs. A bare label is where the model invents: "Case number" once came back as a complete, fictitious German court reference. tr-lint reports that as a number not in the source, which is why its NUM findings come first. And no German draft has yet been reviewed by a translator.

13. Decisions and why

Decision	Chosen	Reason
Model	GaMS3-12B-Instruct for sl↔en; EuroLLM-9B-Instruct-2512 for en↔de	GaMS3 is Slovene-specific; outperforms base Gemma 3 12B on EN→SL and rivals GPT-4o on the Slovene arena. German is absent from its training, so English↔German goes to EuroLLM, which covers every EU official language and has translation in its instruction tuning. The model is chosen by pair in trlib, so a German project cannot fall back to GaMS3 unnoticed
Quantization	Q8_0	Quantization damage is disproportionate for less-resourced languages; throughput is not a constraint
No RAM upgrade	32 GB retained	Workload is bandwidth-bound. More capacity adds no bandwidth and only enables slower, larger models
Segment granularity	Sentence	Matches the translator's review unit and makes the memory reusable across documents
Abbreviation handling	Custom list	Default splitters shatter Slovene legal text at št., čl., odst., which fights the reviewer on every page
Spreadsheet strategy	Unique-string map	Collapses the work and guarantees identical cells translate identically
Verification	Deterministic linter	Catches numeric and consistency errors that models cannot self-detect; costs nothing to run
Back-translation	Dropped	Its value assumed a reviewer reading target-only. Sentence-by-sentence comparison against source catches the same errors directly
Project isolation	Separate memory per matter	Memory holds real sentences. Sharing it across clients would move content between matters
Glossary layering	Shared base + overlay	Terminology is reusable; case specifics are not. Layering gets the benefit without the leak
Data location	Encrypted container	Makes the assistant boundary structural rather than remembered
Dates, amounts, times	Converted, not verbatim	The translator's rule. English takes March 5, 2024 and a decimal point; Slovene takes 5. marec 2024 - ordinal period, lowercase month, spaced - and a decimal comma. Whole-segment values are

Decision	Chosen	Reason
		converted in code, without a model call
Institution names	Translated; source bracketed on first mention per document	A court's name is not an identifier, so it is translated - listing them beside case numbers had made two models read that rule two ways, one leaving Slovene in the output. The source form is kept in parentheses on the first mention in each document and dropped thereafter, because a reader may open any file first and each has to stand on its own
Completed provisions	Flag the context	The model finishes famous provisions from memory. No prompt stopped it and no deterministic check sees it, so segments citing a statute are flagged for word-by-word review
Disk encryption	Container only - accepted	The root filesystem is plain ext4 and stays that way. Retrofitting means re-encrypting in place or reinstalling, and the container is what actually protects the case material at rest. Accepted residual risk, named so it is not rediscovered as a surprise: swap, temporary files, and anything copied out for review are in the clear, as is everything while the container is open. One part of that has since been closed rather than accepted: OCR renders every page of a scan to a PNG, and on an all-scanned corpus that put the whole evidence bundle through /tmp - tmpfs, so RAM-backed and swappable to the plain 8 GB swapfile, and left behind entirely when a run is killed before its cleanup. Those renders now go to <project>/work/tmp inside the container (trlib.case_tmpdir). Encrypted swap is the cheapest of the remaining mitigations if the rest is revisited
Machine-assisted drafts	Permitted, no disclosure	The translator: what counts is the final product, not how it was produced. Most of the trade already edits machine output. No obligation to tell the court
Confidentiality	Premises, not paperwork	Acceptable so long as the texts never leave the translator's premises, and no visitors are received there. Nothing needed in writing
Transfer to translator	USB stick, hand-carried	Machines sit on adjacent tables. The translator judges the crossing not a practical risk; it is the one point where material leaves the container, so it is written down rather than assumed
Billing unit	Source words	What tr-inventory --count reports. Segments are counted alongside because they govern machine time, which is a different question from what is owed
Files not in the source language	Excluded from work and billing	Which is why triage tracks them: by-lang/*.txt locates them so they can be pulled out before

Decision	Chosen	Reason
		translation starts
Quoted provisions	Render only what is present	Never completed from the instrument's official text. Where no established translation exists the source is left with a visible marker, and the translator supplies the wording
Acronym expansion	Footnote, not inline	Expanding KZ-1 in the body harms readability and spacing. A footnote mark carries the full form
Illegible source	OCR_ILLEGIBLE	The translator confirmed the convention; the spelling then changed. [ILLEGIBLE] cannot be selected with a double-click, because word selection stops at the brackets and leaves them behind after a paste - an unwelcome complication for the person replacing every one of them by hand. OCR_ILLEGIBLE selects whole, since underscore is a word character, and unlike _ILLEGIBLE_ it has no leading or trailing underscore for Word or LibreOffice to autoformat into underlining. Override with TR_ILLEGIBLE_MARK
Certification	Batch form, stamp on paper	No per-document certification block. .docx primary, .pdf acceptable, .xlsx for tables since text formats handle them poorly
Language pairs	sl ↔ en and en ↔ de; sl ↔ de not yet	Every one of the three can be source or target. Slovene ↔ German has no model chosen and refuses to translate: the model researched for it is not installed, and pivoting through English doubles the error
Prompt version	Derived from the prompt text	A hand-bumped string invalidated every pair at once and could be forgotten. A hash of the text each pair is actually sent changes exactly when that text does. The two v6 texts keep the name v6, so existing memory stays valid
Reference translations	Reused when identical and agreed	An identical source sentence takes the translator's own rendering with no model call. Only one-to-one pairs whose numbers agree are kept; disagreeing references and OCR-read translations are never reused verbatim. They stay in their project, because memory never crosses matters

14. Open items

- Measure the 7,000-row spreadsheet with `tr-xlsx --survey`. The unique ratio determines whether it is hours or days.
- Exercise a full open/close cycle with a project in the container. The container exists - LUKS2, 40 GB sparse, header backed up to `~/case/header.bak` - has been opened and closed repeatedly with a real project in it, and both guards have been seen to refuse. The bare

mountpoint is immutable so a stray write cannot land there. What remains is a full translate-and-deliver pass, which Phase 1 provides.

- Copy `~/case/header.bak` somewhere off this machine. A corrupted LUKS header means the data is unrecoverable even with the correct passphrase, because the header holds the encrypted master key.
- Calibrate the vision gate on more than two pages. The vision model has earned its place: over eight pages of real scans it agreed with Tesseract on every number except on the one page where fourteen disagreed, and that page was also the one Tesseract itself was least sure of (S5.3). --gate now defaults to 3%, drawn from a 5%-doubtful page that needed the check and 0-2% pages that did not. More pages would firm that line up.
- Establish how often the model completes a statutory provision from memory (S8.1). Two of ten well-known provisions, both ECHR article 6, is enough to know the risk is real and not enough to know its shape.
- Measure reference reuse on a real matter: how many source sentences are reused, and how often pairs.tsv needs correcting (S4.4).
- Have a translator review English→German drafts. EuroLLM is measured for speed and ran without a failed request, but no German output has been reviewed yet (S12).
- Run Phase 1 on a small subset of the real assignment. The translator's preference: real material, so no work is wasted. This is now the gating item - the professional questions are answered and recorded in S13.
- While editing the Phase 1 pages, tally why each edit was made: terminology, register, numbers, additions, omission, formatting. The ratio says whether to proceed; the tally says whether a poor ratio is fixable. Terminology is cheap to fix and there is no glossary yet, so a bad number driven by terminology is a different verdict from the same number driven by register.
- Build the visible marker for quoted provisions with no established translation (S8.1). The trigger needs settling: on a first run the memory is empty, so a naive rule marks every citation.
- Expand acronyms by footnote rather than inline. python-docx has no footnote API, so this means writing the XML directly.
- Locale conversion covers English→German only among the German pairs. Slovene ↔ German and German→English dates and amounts are left to the model, and Slovene ↔ German still needs a model chosen.